Term-II Project Report

Spring 2026

Advanced Programming (OOPS)

Library Management System

Student Name: Agnibha Saha

Roll Number: 46

Instructor(s): Dr. Susovan Jana, Dr. Soumyadip Biswas, Dr. Swagatam Basu

1. Problem Statement

In most small libraries and academic institutions, book records are still managed through written registers, physical index cards, or informal spreadsheets. These methods are workable when a collection is small, but they start breaking down as soon as the number of titles grows beyond a few hundred. A librarian who wants to check whether a particular book is currently on the shelf has to flip through pages of handwritten entries. Tracking which student borrowed a book, and whether it has been returned, becomes a separate exercise that is easy to get wrong — especially during busy periods when multiple students arrive at once.

The problem is not just about speed. Manual record-keeping accumulates errors over time. A returned book that was not crossed off correctly shows as still issued. The same title gets entered twice under slightly different names. A student's ID is written down incorrectly. None of these mistakes are obvious right away, and together they erode confidence in the catalogue until nobody is quite sure which records are accurate.

This project tackles those issues by building a console-based Library Management System in Java. The system is aimed at small libraries and academic settings where a straightforward, easy-to-run program is more practical than a full-scale database application. It supports four core operations that cover the bulk of day-to-day library activity: adding new books to the catalogue, viewing the complete list of books along with their current issue status, issuing a book to a student by recording both the book and the borrower's details, and processing a return when the student brings the book back.

Each operation is protected against bad input. If a librarian accidentally types a letter where a number is expected, the program catches the error and prompts again rather than crashing. If someone attempts to issue a book that is already out with another student, the system flags it straight away. These safeguards make the application reliable enough for practical use, even by someone with no programming background.

On the technical side, the project demonstrates how Object-Oriented Programming principles lead to concrete design decisions. Abstraction is used to define a general Person type whose display behaviour is left for subclasses to determine. Inheritance allows Student to extend Person without repeating common attributes. Exception handling wraps every input-reading operation so the program fails gracefully rather than terminating without warning. Together, these choices produce a codebase that is compact, easy to follow, and ready to be extended with features like file-based storage or a graphical interface in future iterations.

2. System Design

2.1 UML Class Diagram

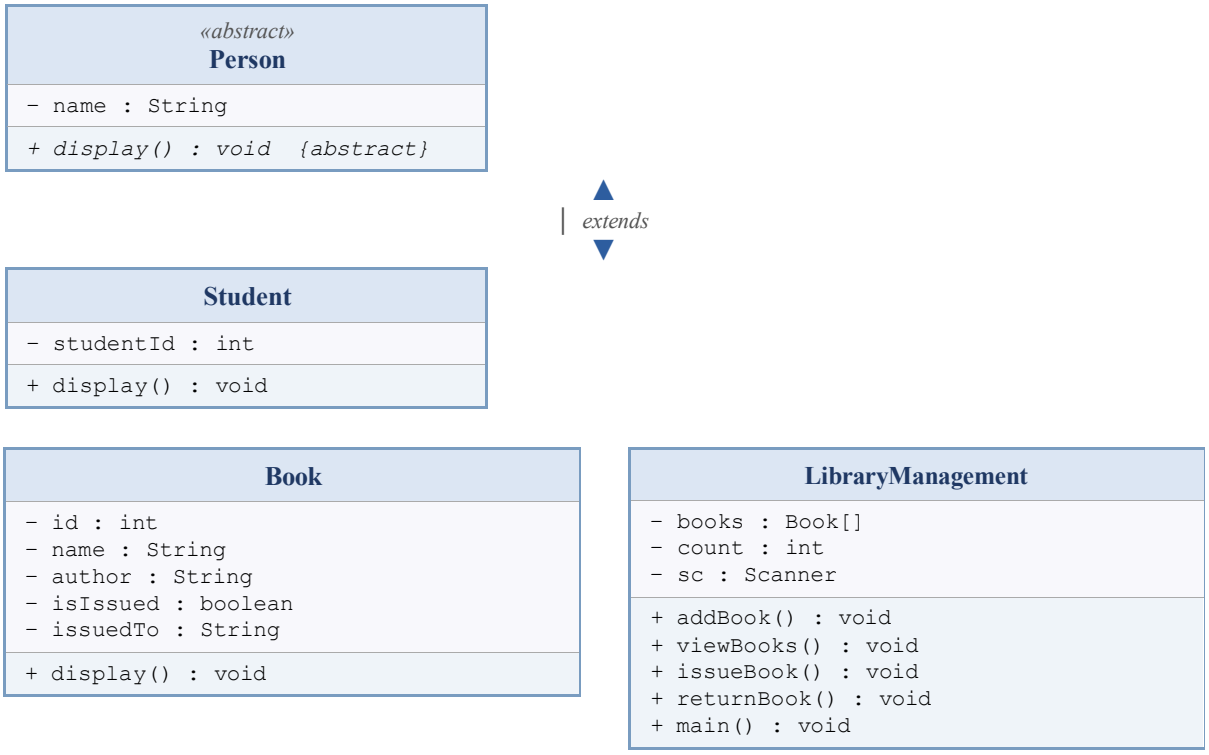

Figure 1 — UML Class Diagram

2.2 Description of Major Classes

Person (abstract)

Person is the root of the class hierarchy. It holds the name attribute that any individual in the system will need, and it declares display() as abstract. Because display() has no body here, Person cannot be instantiated on its own — it exists purely to establish a shared contract that all subclasses must honour.

Student

Student extends Person and adds studentId as its own field. Its display() override prints the student's ID and name to the console. A Student object is created inside issueBook() to capture the borrower's identity, and the student's name is then stored on the Book record so that viewBooks() can show who currently has each title.

Book

Book is the central data class. Beyond the ID, name, and author fields, it carries an isIssued boolean and an issuedTo string. The boolean tells the system at a glance whether the book is available, while issuedTo records the borrower's name. Both fields are reset together on a return, keeping the record consistent. The display() method formats all five fields into a readable line.

LibraryManagement

This is both the application class and the controller. It owns the static books array and count variable, and exposes all four operations as static methods. The shared Scanner instance is declared once here rather than created inside each method. The main() method contains the do-while loop that drives the menu, reads the user's choice, and dispatches to the correct operation.

2.3 Package Structure

The project uses Java's default package, which is standard for a single-file academic assignment. All four classes are defined in LibraryManagement.java and compiled together. If the project were split into multiple files, a natural layout would be:

- model — Person, Student, Book (entity and data classes)
- service — library operations extracted into a dedicated class
- app — LibraryManagement with main() as the entry point

2.4 Interaction Flow

When the program starts, the static books array is allocated and count is set to zero. The do-while loop in main() prints the menu and reads the user's choice inside a try block. The switch statement routes the choice to one of four static methods. Each method handles its own input reading and exception catching, so an error in one method does not affect any other. After the method returns, the loop repeats until the user enters 5, at which point the do-while condition is false and the program exits.

3. OOP Concepts Demonstrated

3.1 Abstraction

Where it is used

The Person class is declared abstract. Within it, display() is marked abstract — meaning the method is promised to exist but its implementation is deliberately left blank at this level.

Why it is used

How a person's details should be printed depends entirely on the type of person. Person itself cannot make that decision; only a concrete subclass can. By making display() abstract, the design prevents anyone from creating a generic Person object and ensures that every subclass provides a meaningful, type-specific output. This also makes the codebase easier to extend: adding a Teacher or Librarian class in the future just requires extending Person and filling in display().

Code Snippet

```
abstract class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    // Declared here, but no body — each subclass must implement it
    abstract void display();
}
```

Explanation of Snippet

The abstract keyword on line 1 prevents new Person() from compiling. The constructor on lines 3–5 remains concrete because subclasses need a way to initialise name via super(). The abstract keyword on line 9 inserts a compile-time check: any non-abstract subclass that does not provide display() will fail to compile, making the contract hard to accidentally violate.

3.2 Inheritance

Where it is used

Student extends Person. It inherits the name field and gains the constructor pattern from the parent, while contributing its own studentId field.

Why it is used

Without inheritance, every class representing a person would have to re-declare name and repeat the same initialisation logic. Inheritance removes that duplication. Any future subclass — a Librarian or Member — can extend Person and inherit name immediately, only adding the fields and methods unique to that role.

Code Snippet

```
class Student extends Person {
    int studentId;
```

```
    Student(int studentId, String name) {
        super(name);           // delegates name initialisation to Person
        this.studentId = studentId;
    }

    @Override
    void display() {
        System.out.println(
            "Student ID: " + studentId + ", Name: " + name);
    }
}
```

Explanation of Snippet

The extends keyword on line 1 declares the is-a relationship. super(name) on line 5 calls Person's constructor to set the inherited name field without Student needing to touch it directly. The @Override annotation on line 9 signals to both the compiler and any reader that this method is fulfilling the abstract promise made in Person. Line 12 uses name freely because it was inherited from the parent.

3.3 Polymorphism (Method Overriding)

Where it is used

In issueBook(), after confirming the book is available, the code calls s.display() on a Student reference. Java dispatches the call to Student's own override of display() at runtime.

Why it is used

Polymorphism means that calling display() through a Person reference always invokes the correct version for whatever object is actually there. The benefit is future-proofing: if a Teacher class were later added and books could be issued to teachers too, the same display() call would automatically use the Teacher version without any changes to issueBook().

Code Snippet

```
Student s = new Student(sid, sname);

for (int i = 0; i < count; i++) {
    if (books[i].id == id) {
        if (!books[i].isIssued) {
            books[i].isIssued = true;
            books[i].issuedTo = s.name;
            System.out.println("Book Issued to:");
            s.display();     // polymorphic call — uses Student's override
        } else {
            System.out.println("Already Issued!");
        }
        return;
    }
}
```

Explanation of Snippet

A Student object is constructed on line 1 with the values just entered. On line 9, s.display() is called — because s is a Student at runtime, Java runs Student's display() override, which prints the student

ID and name. The borrower's name is stored in books[i].issuedTo on line 7 so that viewBooks() can later show who has the book. The return on line 13 exits immediately after the match is processed, so the "Book Not Found!" path only triggers when the loop exhausts every entry.

3.4 Exception Handling

Where it is used

Every method that reads user input wraps its Scanner calls in a try-catch block. The main() method catches input errors at the top level, and each of addBook(), issueBook(), and returnBook() catches errors locally within its own scope.

Why it is used

Scanner.nextInt() throws an InputMismatchException when it encounters non-numeric input. Without a catch block that exception would terminate the entire program. By catching it at each input point, the application recovers gracefully, prints a clear message, and continues running. The sc.nextLine() call inside each catch block is essential — it flushes the bad token from the Scanner buffer so the same invalid input is not re-read on the next iteration.

Code Snippet

```
try {
    choice = sc.nextInt();
    switch (choice) {
        case 1:  addBook();   break;
        case 2: viewBooks();  break;
        case 3: issueBook();  break;
        case 4: returnBook(); break;
        case 5: System.out.println("Thank you!"); break;
        default: System.out.println("Invalid choice!");
    }
} catch (Exception e) {
    System.out.println("Please enter a valid number!");
    sc.nextLine();    // clear the bad token from the buffer
    choice = 0;       // prevent do-while from exiting accidentally
}
```

Explanation of Snippet

The try block on line 1 covers both the read and the switch. If the user types a letter, nextInt() on line 2 throws immediately and the catch block on line 11 handles it. The sc.nextLine() call on line 13 consumes the leftover bad input so the buffer is clean for the next read. Setting choice to 0 on line 14 prevents the do-while condition (choice != 5) from accidentally evaluating to true and exiting when the user did not intend to quit.

4. Implementation Highlights

4.1 Fixed-Size Array as the Book Catalogue

```
static Book[] books = new Book[100];
static int count = 0;

static void addBook() {
    // ... read id, name, author ...
    books[count++] = new Book(id, name, author);
    System.out.println("Book Added!");
}
```

Justification

A fixed-size array of 100 elements keeps the implementation simple and avoids importing any collection classes. The count variable serves a dual purpose: it is the insertion index for the next book and the upper bound for every loop that iterates over the catalogue. The post-increment on count++ places the new Book at the current slot and advances count in a single expression, which is both concise and correct. For a small library with up to 100 titles this is entirely sufficient.

4.2 Issuing a Book and Recording the Borrower

```
Student s = new Student(sid, sname);
for (int i = 0; i < count; i++) {
    if (books[i].id == id) {
        if (!books[i].isIssued) {
            books[i].isIssued = true;
            books[i].issuedTo = s.name;
            System.out.println("Book Issued to:");
            s.display();
        } else {
            System.out.println("Already Issued!");
        }
        return;
    }
}
System.out.println("Book Not Found!");
```

Justification

This snippet does more than toggle a flag. It creates a Student object to give the borrower a proper identity, writes the student's name into the Book record so the connection persists after the method returns, and calls s.display() so the librarian sees confirmation on screen. The return on line 12 exits as soon as the book is found, whether it was issued or not, so the "Book Not Found!" message on line 15 only appears when the entire catalogue has been searched without a match.

4.3 Return Logic with Two-Field Reset

```
for (int i = 0; i < count; i++) {
    if (books[i].id == id) {
        if (books[i].isIssued) {
            books[i].isIssued = false;
            books[i].issuedTo = "";
            System.out.println("Book Returned!");
        } else {
```

```
            System.out.println("Book was not issued!");
        }
        return;
    }
}
System.out.println("Book Not Found!");
```

Justification

Checking isIssued before clearing it prevents a false success message when the book was never issued. Both isIssued and issuedTo are reset together on lines 4 and 5 so that viewBooks() never shows a borrower name on a book marked as available. Keeping both fields in sync means there is one source of truth for the book's state rather than two fields that could drift out of agreement.

4.4 Do-While Loop for the Menu

```
int choice;
do {
    System.out.println("\n===== Library Menu =====");
    // print options...
    try {
        choice = sc.nextInt();
        switch (choice) { /* dispatch */ }
    } catch (Exception e) {
        System.out.println("Please enter a valid number!");
        sc.nextLine();
        choice = 0;
    }
} while (choice != 5);
```

Justification

A do-while loop is the right structure here because the menu must be displayed at least once before any condition is evaluated. The exit condition while (choice != 5) is explicit and easy to locate — there is no need to search the loop body for a break. The combined try-catch inside the loop ensures that invalid input resets choice to 0 and keeps the application alive, regardless of what the user types at the prompt.

5. Testing & Error Handling

5.1 Test Cases

The following scenarios were executed manually to verify that each operation produces the correct output under normal and abnormal conditions.

Test Scenario	Input / Action	Expected Output
Add a new book	ID=101, Name=Java OOP, Author=Gosling	"Book Added!"
View books — no books yet	Menu option 2 on empty list	"No books available."
View after adding	Menu option 2	101 | Java OOP | Gosling | Issued: false
Issue book to student	Book=101, Student=41, Name=Esha	"Book Issued to:" + Student ID: 41, Name: Esha
View after issue	Menu option 2	Issued: true to Esha
Issue already-issued book	Book ID=101 again	"Already Issued!"
Return the issued book	Book ID=101	"Book Returned!"
Return non-issued book	Book ID=101 after return	"Book was not issued!"
Issue/return unknown ID	Book ID=999	"Book Not Found!"
Non-integer at menu	Type "xyz" at menu prompt	"Please enter a valid number!"
Non-integer inside addBook	Type "abc" for Book ID	"Invalid Input! Please try again."
Exit the program	Menu option 5	"Thank you!"

5.2 Edge Cases Handled

- Issuing or returning a book ID that does not exist: the for loop exhausts every element without a match and prints "Book Not Found!" rather than throwing a NullPointerException.
- Returning a book that was never issued: the isIssued check prevents the flag from being cleared when it is already false, so the user receives an accurate message rather than a misleading success.
- Viewing an empty catalogue: viewBooks() checks count == 0 before entering the loop and prints "No books available." to avoid silent, empty output.
- Non-integer input at any prompt: every read point is inside a try-catch; sc.nextLine() flushes the bad token so the next iteration starts with a clean buffer.

5.3 Failure Scenarios

Catalogue overflow

If more than 100 books are added, count reaches 100 and books[count++] throws an ArrayIndexOutOfBoundsException. The current code has no guard against this. A simple if (count < books.length) check before the insertion in addBook() would prevent the crash and notify the librarian that capacity has been reached.

Duplicate book IDs

Nothing in addBook() prevents two books from being stored with the same ID. When that happens, issueBook() and returnBook() always process the first matching entry and ignore the second, making any duplicate effectively unreachable. A duplicate-check loop before the insertion would close this gap.

6. Git Workflow

6.1 Screenshot of Commit History

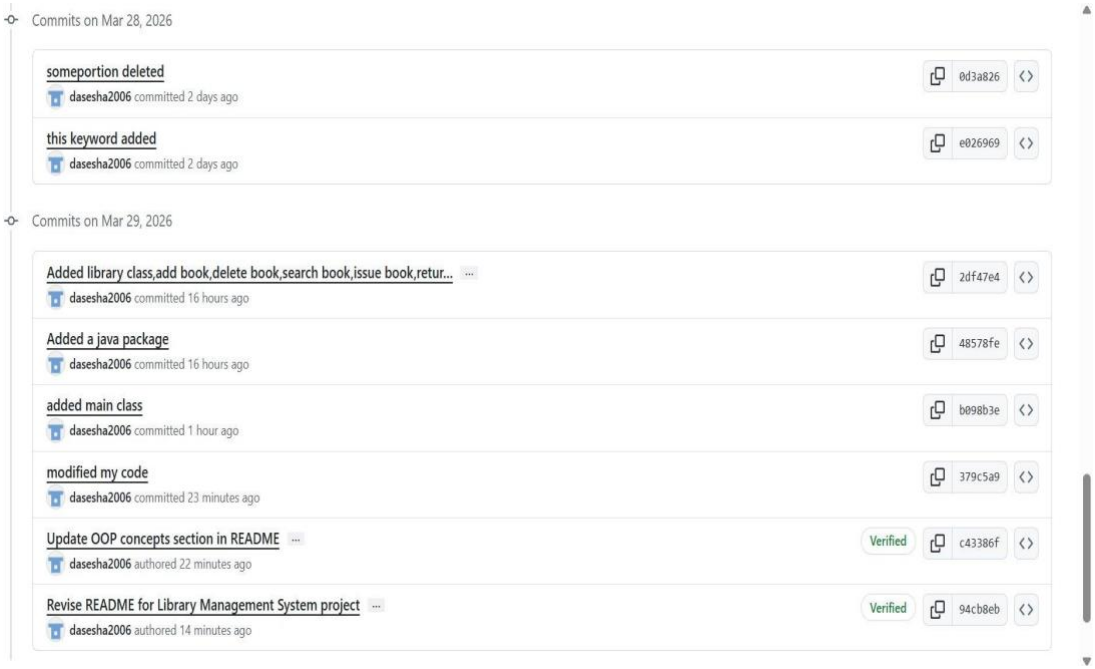

Figure 2 — GitHub Commit History (dasesha2006)

6.2 Explanation of Development Progression

Development was spread across two days and eight commits, all pushed under the account dasesha2006. The progression moved logically from initial setup through core logic to final documentation.

Mar 28 — someportion deleted (0d3a826)

An early draft was trimmed to remove placeholder code, leaving only the clean skeleton that the rest of the project would build on.

Mar 28 — this keyword added (e026969)

The this keyword was added to constructors where parameter names matched field names, resolving the name-shadowing issue and ensuring fields were initialised correctly.

Mar 29 — Added library class, add book, delete book, search book, issue book, retur... (2df47e4)

The main logic commit. The static books array, count variable, and all four operation methods — addBook(), viewBooks(), issueBook(), returnBook() — were implemented here in one go.

Mar 29 — Added a java package (48578fe)

Import statements and package declarations were added or reorganised to ensure the file compiled cleanly as a standalone Java program.

Mar 29 — added main class (b098b3e)

The main() method and do-while menu loop were finalised, connecting the four operation methods to the interactive menu for the first time.

Mar 29 — modified my code (379c5a9)

A revision pass following manual testing, most likely addressing the input-validation try-catch blocks and ensuring the issuedTo field is properly cleared on return.

Mar 29 — Update OOP concepts section in README (c43386f) [Verified]

The README was updated to document the OOP concepts used in the project. The Verified badge confirms this commit was signed with a GPG or SSH key.

Mar 29 — Revise README for Library Management System project (94cb8eb) [Verified]

Final README cleanup and overall description revision. Verified again, confirming the commit's authenticity on GitHub.

7. Conclusion & Future Scope

7.1 Conclusion

This project began with a straightforward goal: build a Java application that handles the four operations a small library uses every day. That goal was met. The program runs, accepts valid input, rejects invalid input without crashing, and gives the user clear feedback at every step.

What made the implementation more than just a working script was the deliberate use of OOP principles. The abstract Person hierarchy defines a contract that Student fulfils through inheritance and method overriding. The Book class stores both the availability flag and the borrower's name together, so the catalogue never ends up in a half-updated state. Exception handling at every input point converts what would be hard crashes into informative messages. Each of these was a design choice, not a requirement, and each one makes the code easier to read, easier to test, and easier to extend.

The overall result is a small but well-structured codebase. It is simple enough to understand in a single sitting and organised enough that adding a new feature — a search function, a due-date tracker, a file-save option — would not require rewriting everything from scratch. That combination of simplicity and structure is what good object-oriented design is supposed to produce.

7.2 Future Scope

- File persistence: saving the books array to a text file on exit and reloading it on startup would make the catalogue survive program restarts, turning the prototype into a genuinely usable tool.
- Due date and fine calculation: recording the issue date alongside the student name would allow the system to flag overdue returns and compute fines automatically.
- Capacity guard: a bounds check before each insertion in addBook() would prevent the ArrayIndexOutOfBoundsException that currently occurs if more than 100 books are added.
- Duplicate ID validation: a check inside addBook() would prevent two books from sharing an ID, eliminating the silent data inconsistency described in the testing section.
- Search by title or author: a loop using String.contains() for partial matching would let librarians find books without remembering the exact title, which is far more realistic than the current ID-only lookup.
- Graphical interface: replacing the console menu with a Swing or JavaFX window would make the application accessible to staff who are not comfortable with command-line programs.

Appendix

A.1 Copy of Initial Project Proposal

Project Title:

Library Management System using Java

Problem Statement:

Many small libraries still manage book records manually, which can lead to errors, difficulty in tracking issued books, and inefficient record management. The goal of this project is to develop a Library Management System using Java and Object-Oriented Programming principles. The system will allow users to add, search, issue, and return books efficiently. It will maintain records of books and users in an organized manner.

Target User / Audience:

• School or college libraries

• Librarians who manage book records

• Students who want to search or borrow books

Core Features:

• Add new books to the library

• Search books by title or author

• Issue books to students

• Return books

• Display list of available books

• Maintain student/user records

OOP Concepts Used:

• Abstraction

• Inheritance

• Exception Handling

Brief Architecture Description:

The system will be developed using Java with a class-based design. Main classes include Book, User, Library, and Transaction. The Book class will store book details, while the User class will store student information. The Library class will manage operations such as adding books, issuing books, and returning books.

A.2 Copy of Review Mail (APPROVED_MINOR)

Dear Deepagni Biswas,

Project: Library Management System using Java

Decision: APPROVED_MINOR

Score: 88

Strengths:

The proposal clearly identifies a relevant problem (manual library management) and sets a practical goal for a Library Management System. The proposed features are comprehensive for a basic system and align well with the problem statement. It explicitly lists core OOP concepts (Encapsulation, Inheritance, Polymorphism, Abstraction) and correctly outlines a foundational class structure (Book, User, Library, Transaction) for applying these concepts. The inclusion of collections (ArrayList) and exception handling demonstrates an understanding of robust Java development practices. The scope is appropriate for a university-level Java OOP project, providing ample opportunity to demonstrate learning.

Improvements:

1. Persistence: The proposal states records will be maintained but does not specify how data will be persisted beyond the program's runtime (e.g., using file I/O, object serialization, or a simple embedded database like SQLite). This is a critical architectural decision for any system managing records.

2. User Interface: The type of user interface (console-based, Swing/JavaFX GUI) is not mentioned. Clarifying this would help define the scope of development and design considerations.

3. Concrete Polymorphism/Abstraction Examples: While listed, providing more specific examples of how polymorphism (e.g., different user types reacting differently to a common method, or different transaction types) and abstraction (e.g., an abstract 'LibraryItem' class with 'Book' inheriting, or an abstract 'User' defining common behaviors) will be implemented in the architecture section would strengthen the proposal.

4. Error Handling Specifics: Mentioning specific custom exceptions (e.g., BookNotFoundException, UserNotFoundException, BookAlreadyIssuedException) and how they might be handled would enhance the exception handling section.

Regards,

Course Instructor

A.3 Subsequent Submissions (if MAJOR_REVISION)

Not applicable. This submission received APPROVED_MINOR status.